McMaster University ENGINEERING | mohawk | W BOOTH SCHOOL OF ENGINEERING PRACTICE AND TECHNOLOGY
McMaster-Mohawk Bachelor of Technology Partnership

Laboratory 6
Introduction to Stepper Motors

SMRTTECH 3CC3

Cloud Computing and Internet of Things

B. Tech - SMRTTECH

PRELAB

1. Please download & install the Arduino IDE from the official website: https://www.arduino.cc/en/main/software

Introduction

A stepper motor is simply a motor which can be controlled to take small increments of a rotation and then stop. In addition to excellent position control, stepper motors have what is called holding torque. Many types of electrical motors, when not spinning or moving, offer very little torque to whatever the shaft is connected. A stepper motor, however, has a high torque. When the shaft is not rotating, but the coils are energized through the controller circuitry, it will be difficult to rotate the shaft of the stepper motor with your hand. Try it.

The simplest model for explaining the operation of a stepper motor is to assume a simple two winding model with a permanent magnet core. In Figure 1 a simple diagram of the stepper motor can be seen. The windings 1A and 1B, when energized, have opposite magnetic polarities. Figure 2 shows the 4 sequential steps to produce 4 steps or incremental turns of the shaft, keep in mind that it may take anywhere from 180 to 2000 such steps to produce a full revolution of the shaft.

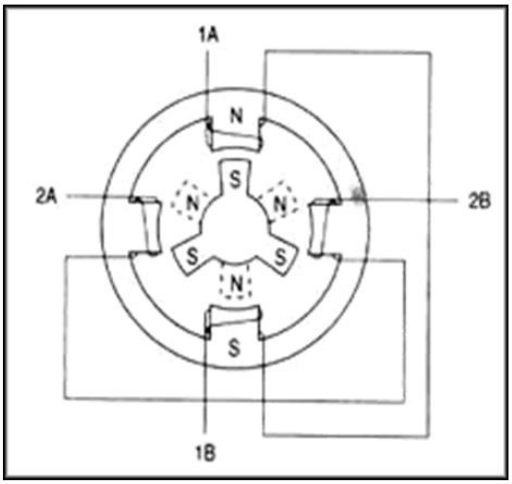


FIGURE 1: BASIC STEPPER MOTOR

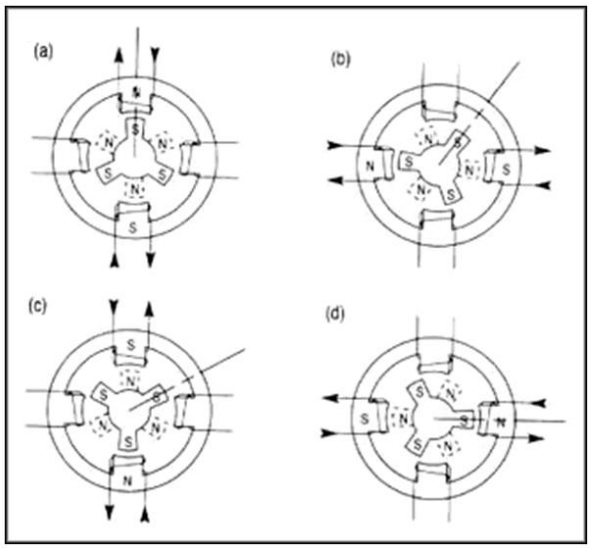


FIGURE 2: SEQUENCE TO PRODUCE MULTIPLE STEP

By energizing and de-energizing the windings at certain intervals and rates, it is possible to take less then a full increment or step. The detail will not be covered here, but the procedure is referred to as micro-stepping.

OBJECTIVES

In this lab you will learn the following:

1. Learn how to build and drive **Stepper Motors** using LabVIEW and Arduino.
2. Learn how to **compile and load firmware** into the Arduino microcontroller.
3. Learn to receive and parse data over **serial** between LabVIEW & Arduino.

MATERIALS

1. Jumper Wires
2. Stepper Motor
3. Stepper Motor Driver
4. Arduino Mega
5. External Power Supply
6. External Power supply board

PROCEDURE

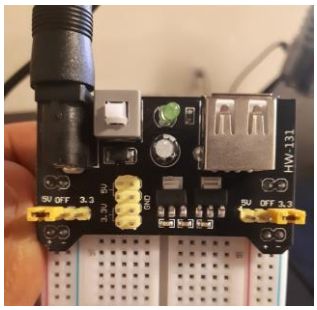
Exercise 1: Stepper Motor Setup

WARNING: THE STEPPER MOTOR MAY HEAT UP DURING OPERATION. THIS IS NORMAL BUT BE CAREFUL WHEN YOU PICK UP THE MOTOR AFTER IT HAS BEEN IN OPERATION.

1. Use a screw driver to adjust the voltage output of the power adaptor to 9V.

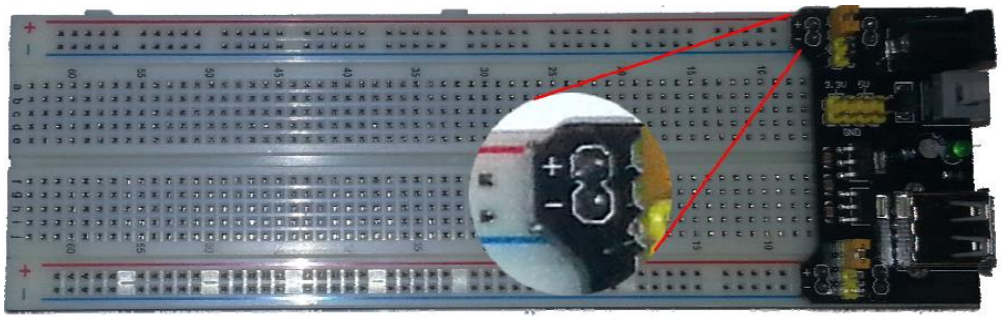


2. Connect the Breadboard Power Supply board shown below to your breadboard. This board will provide a regulated Output voltage of 3.3V/5v. In this lab, only the 5V output will be used to power the motor. Make sure the yellow jumpers are as shown below.



Important note1:

A. The 5V output of the Arduino will not be used to power the motor because the Arduino cannot supply high motor currents. Drawing a high current from the Arduino board might damage the board.
B. Make sure that you align the module correctly on the breadboard. The negative pin(-) on module lines up with the blue line(-) on breadboard and that the positive pin(+) lines up with the red line(+). Failure to do so could result in you accidently reversing the power to your project.

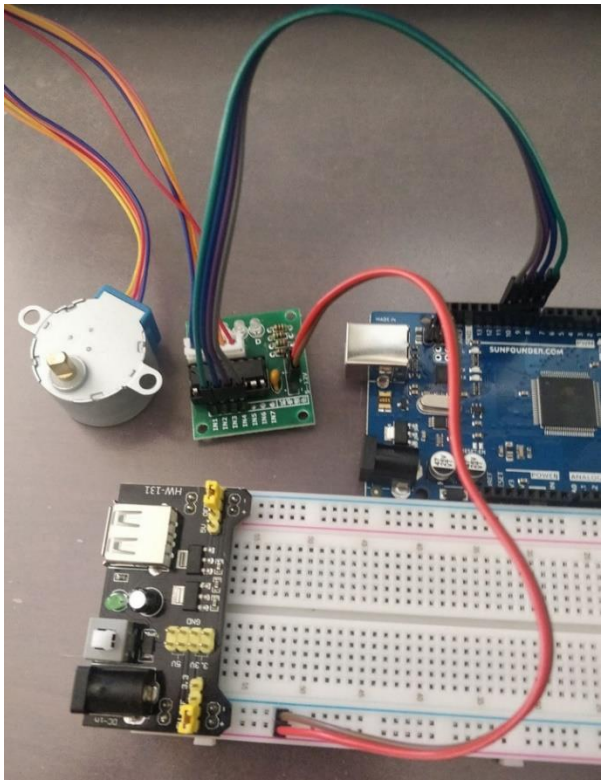


3. Connect the power adaptor outlet to the barrel jack connector of the Breadboard Power Supply as shown below.



4. Build a circuit using your **Stepper Motor**, **Stepper Motor driver**, **External Power Supply** and **Power Supply Board** as shown in **Figure 1**. The motor driver uses the 5V regulated voltage produced by Breadboard Power Supply to provide a high amount of current that the Arduino itself cannot provide.
    a. Make sure the digital out pins connect with the correct driver pins:
        i. Digital (8) Write -> 1N1
        ii. Digital (9) Write -> 1N2
        iii. Digital (10) Write -> 1N3
        iv. Digital (11) Write -> 1N4
    b. Connect one of the GND pins of the Arduino to the GND rail of the breadboard. They must share a **common ground**.
    c. Make sure that you connect the **5V** from the red rail of the bread board to the **+ pin** of the 5-12V pins of the motor driver
    d. Make sure that you connect the **GND** from the blue rail of the bread board to the **- pin** of the 5-12V pins of the motor driver
    e. Make sure the Power Supply board is turned **ON** when using the motor. There is a white button on the board that must be pressed to turn the board on. An LED will light up indicating the board is ON.

End of Exercise



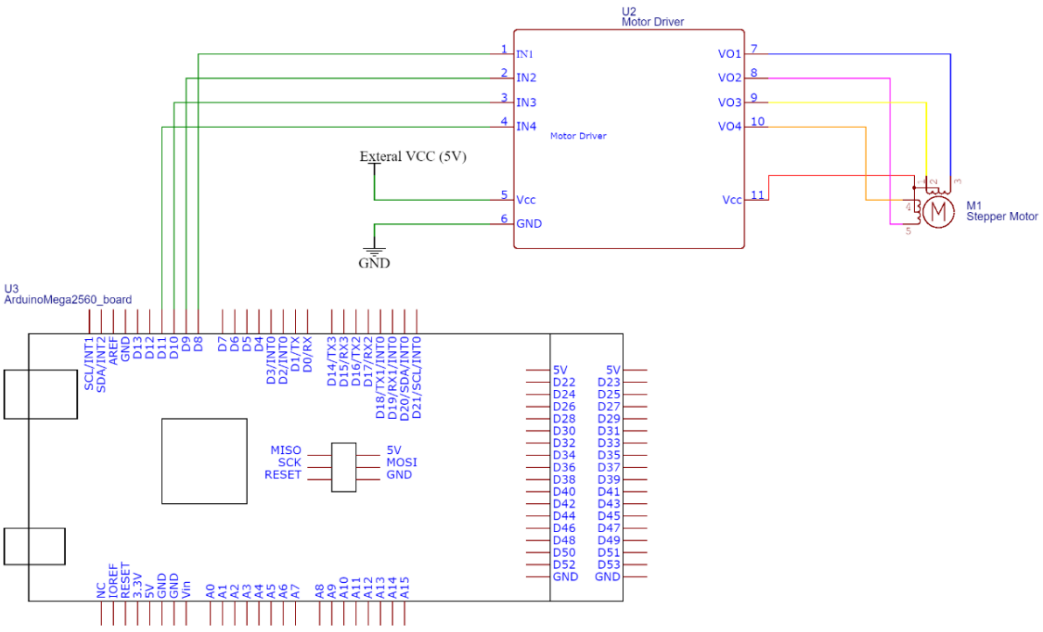
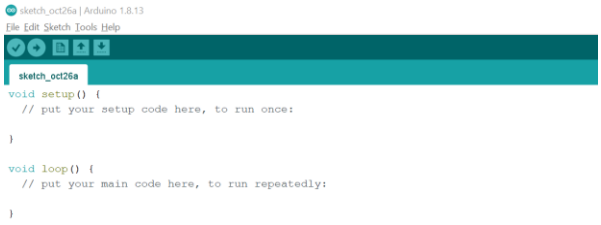


Figure 1: Stepper Motor Setup

Exercise 2: Arduino IDE

1. Launch the Arduino IDE. Select **File>>New**.



2. Copy and paste the following code into the IDE.

```
#include <Stepper.h>

// Parsing Variables
String command;
String desSpeed;
String desRevs;
String desDirection;

int desSteps = 0;
int firstIndex = 0;
int secondIndex = 0;

// define the stepper motor and parameters
const int STEPS_PER_REV = 2048;
Stepper myStepper = Stepper(STEPS_PER_REV, 8, 10, 9, 11);

void setup() {
  Serial.begin(9600);
}

void loop() {

  command = "";

  if (Serial.available()>0)
  {
    command = Serial.readStringUntil('\n');

    //parse the command
    firstIndex = command.indexOf(',');
    secondIndex = command.indexOf(',',firstIndex + 1);

    desSpeed = command.substring(0,firstIndex);
    desRevs = command.substring(firstIndex+1,secondIndex);
    desDirection = command.substring(secondIndex+1);

    //actuate the stepper motor
    desSteps = desRevs.toFloat()*STEPS_PER_REV*desDirection.toInt();
    myStepper.setSpeed(desSpeed.toInt());
    myStepper.step(desSteps);
  }
}
```

3. The Arduino programming language has the exact same syntax as C/C++. For a quick guide refer to https://www.arduino.cc/reference/en/ .
    a. Notice the #include <Stepper.h> preprocessor directive. We will use this standard Arduino stepper library to actuate the motor.
    b. Notice the void setup() function. This function is executed once at the bootup of the Arduino. We use this function to initialize various parameters such as serial communications, motor speed, and local/global variables.
    c. Notice the void loop() function. This function is the main function. It will continuously loop while the Arduino is on. The Arduino is controlled in real time within this function.
    d. In our main loop we will be reading the serial input buffer, parsing the input string from the buffer, and actuating the stepper motor based on the input.

4. Connect the Arduino to the serial port through USB.

5. Configure the Arduino software
    a. Under **Tools»Board** select **Arduino Mega**
    b. Under **Tools»Port** select the port connected to the Arduino

6. Click **Verify**  and make sure there are no complication errors.

7. Click **Upload**  to load the firmware onto the Arduino.

8. **Test** the code functionality using the built-in **serial monitor** by selecting **Tools>>Serial Monitor.** You must use the following syntax to specify the RPM, number of revolutions, and direction of rotation:

    RPM,*Number Revolutions*,*Direction*

    Where:

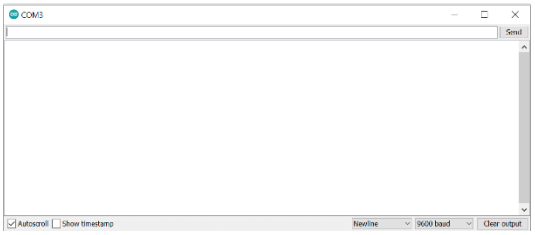
    RPM is any integer (Try playing around with different RPMs. RPM 5-15 worked well for my stepper. Anything slower will take a very long time to complete. Anything faster would lock the motor)

    Number Revolutions is a floating-point value.

    Direction is either 1 (clockwise) or -1 (counter clockwise)

    For example: `10,1.5,1` means RPM=10, Number revolutions=1.5, Direction=Clockwise

    a. Click on **Tools»Serial Monitor**. A window like the one below will open.



b. Make sure the **Baud Rate** is set to 9600. This matches the baud rate specified in the setup() function.

c. Make sure the External Power is ON.
d. Type in `10,1.5,1` and hit send. Your stepper motor should move accordingly.
e. Try several more times with different parameters.
f. When done make sure to **CLOSE** the serial monitor.

End of Exercise

Exercise 3: LabVIEW Serial Communication

In this exercise you will use LabVIEW nodes to send data over your serial port to actuate the stepper motor. The data sent will be in the same format as outlined in Exercise 2.

1. Open a new VI and go to the **Front Panel**.

2. Add an **OK Button**, **Stop button**, **Numeric Control**, and **Vertical Toggle Switch**.
    a. Name the OK Button `Message Arduino`.
    b. Name the String Control `Revolutions`.
    c. Name the Vertical Toggle Switch `Direction`. Add two floating labels. The true case should be labelled `Clockwise` and the false case labelled `Counter clockwise`

4. Add a **Meter** from **Controls»Modern»Numeric**. Click on the left most value and change it to 5. Click on the right most value and change it to 20. You Front Panel should look like **Figure 2**.

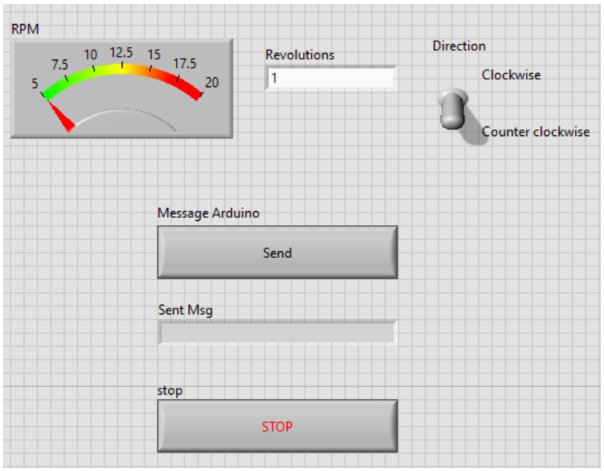


Figure 2: LabVIEW Front Panel

5. Go to the **Block Diagram.**

6. Add the **Configure Port**, **Set Buffer Size**, **Write** and **Close** functions from **Functions»Data Communication»Protocals»Serial.**
    a. Right click on the `VISA Resource Name` node of the **Configure Port** Function and add constant. Set the constant to your Serial Comm Port.

7. Add a **While Loop**, **Wait** and **Case Structure**. Your `RPM`, `Revolutions` and `Direction` functions should be inside the `True` case. In the false case, just carry the wires through. Wire your Functions as shown in **Figure 3**.

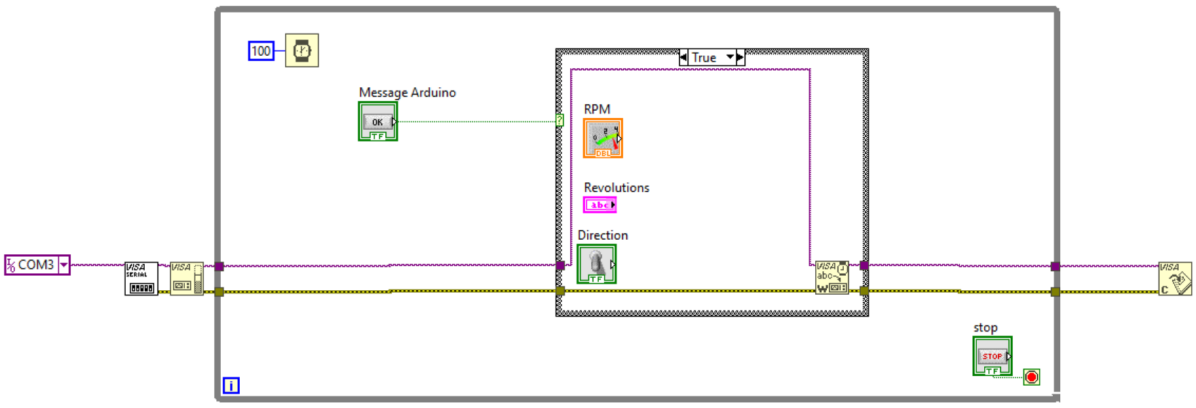


Figure 3: Block Diagram

8. Add a **Concatenate Strings** function from **Functions»Programming»String**
    a. Drag down to expand the number of nodes on the function. You will need 5 input nodes:
        i. Node 1 = RPM (in string format)
        ii. Node 2 = **String constant ,**
        iii. Node 3 = Revolutions
        iv. Node 4 = **String constant ,**
        v. Node 5 = Direction
    b. Nodes 2 & 4 are the string delimiters.
9. Add a **Number to Decimal String** function from **Functions»Programming»String»Number/String Conversion.** Change the **Meter** (RPM) from Indicator to Control and wire its output to the `Number` input node of **Number to Decimal String**

10. Add a **Select** function. Set the true case to **String Constant** `1` and the false to `-1`.

11. Add an **Indicator** to the output node of the **Concatenate String Function**. This way you can see exactly what string is being sent to the Arduino.

12. Wire the functions as shown in Figure 4.

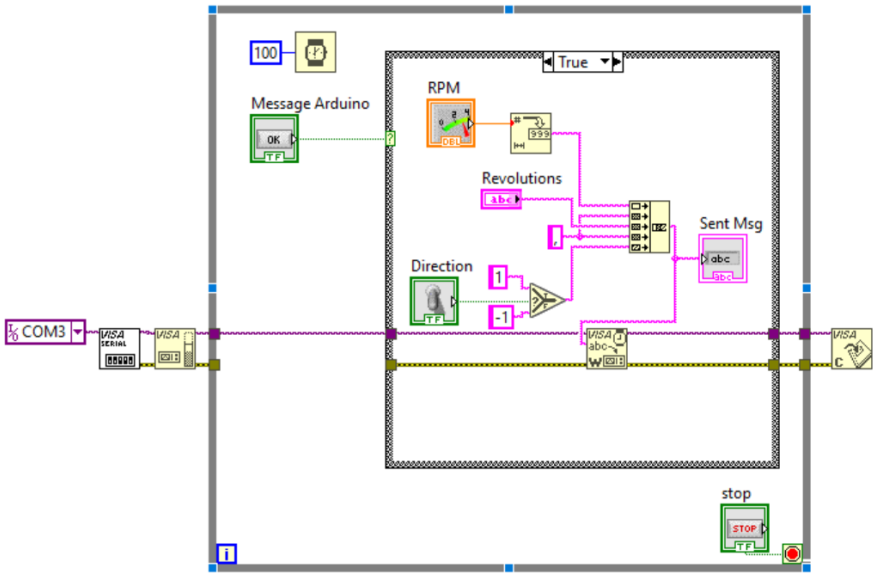


Figure 4: Complete LabVIEW Block Diagram.

13. Test your program you should be able to send commands over serial to the Arduino. Make sure the **Serial Monitor** window from the previous exercise is **closed.** By default, only one client can use the same port at once.

End of Exercise

EXTEND YOUR KNOWLEDGE

1. Replace the **Revolutions Control** with an **Angle Control.** For example, rather than specifying 0.5/1/2 Revolutions, you should type in 180/360/720 degrees.

SUBMISSION

1. Explain in detail the Arduino Code that is provided.
2. Explain how did you modify your code in the extend your knowledge section.
3. What is the maximum speed that the motor can have?
4. When the motor is not moving, try to move the motor with your hand, what is your observation, why?
5. What is the purpose of the motor driver?
6. Why an external power supply is used in this lab instead of the 5V of the Arduino board.
7. In this lab, you have programmed your Arduino using the Arduino IDE and the data were exchanged between the Arduino and LabVIEW using VISA Serial. Explain the possible advantages and disadvantages of using this approach versus using the LINX library that was used in the last lab (Laboratory 5).